PYC

exec

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Aqui está a análise passo a passo do arquivo fornecido (que consiste em um trecho de um arquivo compilado de bytecode Python `.pyc` com técnicas de ofuscação):

1. Análise Estrutural do Arquivo

Formato: O arquivo contém assinaturas típicas de arquivos `.pyc` (Python Compiled), estruturados para armazenar bytecode executável pelo interpretador Python.

Módulos Importados: Foram identificados os módulos padrão `base64` e `zlib`. Esses módulos são amplamente utilizados em scripts legítimos, mas em contextos de segurança da informação, também formam a base clássica para descompactação de payloads em tempo de execução.

2. Análise de Padrões e Ofuscação

Nomes Aleatórios (Garbage Identifiers): O código faz uso de identificadores e nomes de variáveis gerados por algoritmos automáticos de ofuscação (ex: `dfhulxliqohxamy`, `osatiehltgdbqxk`, `wtqiceobrebqsxls`, `idzextbcjbgkdih`). Essa técnica visa dificultar a leitura humana e a análise estática por analistas de segurança.

Nomenclatura Baseada em Hash: O script faz referência interna a um arquivo nomeado com um hash hexadecimal (`C566bb30ad42762b4584c55267158ea901602fc149fb2e0911107e49432de1249.py`), indicando que o script original foi gerado ou renomeado por uma ferramenta automatizada de build ou empacotamento.

Payload Oculto: É visível uma sequência de dados comprimidos e codificados (iniciada com `eNq...`), que corresponde a uma string em Base64 compactada via

`zlib` .

3. Mecanismo de Execução

O fluxo lógico do script desempacota o payload em memória utilizando `base64.b64decode` e `zlib.decompress` , repassando o resultado diretamente para funções de execução dinâmica de código (como `exec`).

Veredito Final: É benigno ou malware?

Este arquivo é classificado como MALWARE (ou um *loader/dropper* malicioso).

Justificativa:
Nomes de variáveis aleatórios combinados com empacotamento via `base64 + zlib` e execução dinâmica (`exec`) formam uma assinatura comportamental clássica de **ofuscação maliciosa**. Essa estrutura é utilizada quase exclusivamente por agentes maliciosos para ocultar o código real (payload) de varreduras estáticas de antivírus, sistemas EDR e ferramentas de inspeção de código antes que o artefato seja executado

Qual é a categoria principal deste malware?

Categoria Principal do Malware

A categoria principal deste ficheiro é a de **Loader** (também designado por **Dropper** ou **Stager**), integrando-se num contexto mais amplo de **Ataque à Cadeia de Fornecimento (*Supply Chain Attack*)**.

Justificação Técnica

Função de Carregamento (*Loader*/*Stager*): O artefacto em bytecode Python (`.pyc`) não concentra toda a carga maliciosa na sua estrutura estática. O seu único objetivo funcional é desofuscar, descomprimir (recorrendo a `base64` e `zlib`) e executar dinamicamente o *payload* real em tempo de execução.

Contexto de Ameaça: Esta arquitetura é amplamente utilizada em campanhas direcionadas a ambientes de desenvolvimento e repositórios de software (como campanhas de comprometimento de dependências e pacotes), servindo para descarregar ferramentas de acesso remoto (RATs) ou *infostealers* de forma furtiva.

Em suma, trata-se de um componente focado na **evasão e no carregamento remoto**, desenhado para ocultar a verdadeira natureza da ameaça perante análises estáticas iniciais.